# Verifiable incremental matrix maintenance

Database use case and method overview

## (a) Database use case

Which soil types occur at each elevation?

New Covertype records · illustrative rows

| Record | Soil type | Elevation band |
|---|---|---|
| $r_1$ | 2 | 2,400–2,500 m |
| $r_2$ | 1 | 2,600–2,700 m |

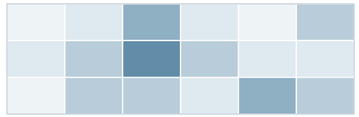
↓

Randomized reports $U_j$, $V_j$

Fresh coins after program fixation

↓

Accepted deltas → decode and debias



Estimated co-occurrence $\hat{H}$ · 40 × 40

## (b) Reuse vs. recomputation

Full recomputation

$$C_{new} = A_{new}B$$

$A_{new}$ × $B$ = $C_{new}$

### Incremental maintenance

$$A_{new} = A_{old} + UV^T$$

$$C_{new} = C_{old} + U(V^TB)$$

$C_{old}$ + $\Delta C$ → $C_{new}$

cached · fresh delta · updated

Separate dense-GPU benchmark · checked updates

161.93 → 83.16 ms (1.95×)

n = 4096 · r = 8 · t = 2 · q = 65521

## (c) Stateful verification

**Outsourced compute service** | **Database verifier**

Retain arbitrary paid history and cached state

**1 Before fresh inputs** | **2 Before random checks**

**Fix program**
$F_j$, $w_j$
and coefficients

**Disclose**
$U_j$, $V_j$

**Evaluate fixed program**
Freeze the full candidate

**Draw independent, uniform check vectors**
$z_h$, $h = 1, \ldots, t_j$

Candidate

$$\tilde{C}_j = \hat{C}_{j-1} + F_j(U_j, V_j)$$

Check every vector

$$(\tilde{C}_j - \hat{C}_{j-1})z_h = U_j(V_j^T(Bz_h))$$

**All checks pass?**

Yes → Accept: advance state

No → Reject: end sequence; no advance

Retain state for next round

### Work-dependent bounds

Rank-conditioned agreement for uniform factors.
Adaptive composition under a pathwise work budget.

### Checking-budget allocation

Exact heterogeneous threshold allocation.
Predetermined checks; equal per-check costs.

### Database workloads

Structured row updates: reuse benchmark.
Randomized reports: nonuniform work guarantee.

Scope: admitted homogeneous bilinear programs, conditional freshness, and both causal boundaries; trusted kernel admission, not arbitrary-host attestation.